

FASERCAL V10

Install, Build, Run — a consolidated reference

André Rubbia

IPA Rubbia Group, ETH Zürich

October 2, 2026

Outline

- 1 Pipeline overview
- 2 Known issue: proton-fraction bug
- 3 Known issue: muon-spectrometer field consistency
- 4 Install from scratch
- 5 Build
- 6 Repository layout
- 7 Where the data lives
- 8 Python wrapper scripts
- 9 Running it
- 10 Further reading

- **ConvertGENIE / ConvertFASERMC**: convert a GENIE or official FASERMC neutrino-interaction sample into the common `TP0Event` format
- **FASERG4** (`faserps`): runs that through Geant4, writes per-event `TcalEvent` truth files
- **Batch** (`batchreco.exe`): reconstructs truth files into `TP0RecoEvent` files
- **EvDisplay** / analysis / **TauSearch**: consumes the reconstructed output
- `run_regression_tests.py`: wraps the whole chain end to end for an automated simulate → reconstruct → compare check

Known issue: FASERG4 scintillator proton fraction ($V9 \rightarrow V10$)

- FASERG4's scintillator material definition (3DCal tracker layers) had the wrong free-proton (H) vs. carbon-12 (C) content — fixed between production versions **V9** and **V10**
- **Root cause:** `G4Material::AddElement()` is overloaded — one variant takes an atom count (`G4int`), another a mass fraction (`G4double`). Declaring the stoichiometry counters as `G4double` silently selected the mass-fraction overload, so atom counts were read as mass fractions
- **Symptom:** free-proton and carbon-12 targets interacted at almost the same rate in V9 ($\approx 1:1$) instead of the physically expected $\approx 1:12$ ($\propto$ mass number A); V10 correctly gives $\approx 12:1$
- **Effect:** neutrino-induced CC/NC rates rise $\sim 15\text{--}18\%$ (a more isoscalar target supplies more neutrons for the neutrino-preferred d -quark coupling), antineutrino rates stay roughly flat; overall 3DCal fiducial total $+4.4\%$
- also shifts scintillator density ($1.03 \rightarrow 1.06 \text{ g/cm}^3$), hadronic interaction length, and dE/dx (Z/A change); Birks' constant has *not* yet been re-checked against the new composition
- full analysis: `FASERflux/faserg4_proton_fraction_bug.tex` (Fabio Cufino, André Rubbia)

Known issue: muon-spectrometer field consistency

- **Motivation:** the muon spectrometer's magnetic field was implemented *twice* — once in Geant4 simulation (`MuonMagneticField`) and once in GenFit reconstruction (`GenMagneticField`) — two independent hand-written implementations kept in sync only by hand, with no test proving they agreed and no check that either matched the real detector geometry
- **Fix:** extracted one shared, framework-independent function, `FASER::ComputeMuonSpectrometerField()`; both G4 and GenFit now call it as thin unit-converting adapters, so the two field *formulas* can no longer silently diverge — `MuonSpectrometerFieldTest (gtest)` checks G4 and GenFit agree
- **Real bug caught by the test:** outside the field's modelled envelope, GenFit's plain “Magnet” branch fell through to a stray non-zero fallback (0.001 kG in *Y*) where G4 returned an exact zero — a genuine, if tiny, pre-existing sim/reco divergence
- **Second problem:** the slit position deciding where the field flips sign / cuts to zero was a bare hardcoded literal duplicated in **four** places, agreeing only by convention, not construction — a future geometry change could silently desynchronize the field model from the geometry
- **Fix:** `MagnetGeometryProbe` samples the *actual* `TGeoShape` loaded from the real GDML to find where the iron truly ends and the slit truly begins, so reconstruction derives the slit position live instead of trusting a literal; `MagnetGeometryProbeTest` unit-tests the probe itself, including a case that deliberately mismatches the geometry to prove a future drift would actually be caught

1. Install from scratch

```
git clone https://github.com/rubbiaa/FASER.git
cd FASER
source setup.sh
```

- `setup.sh` auto-detects the site (André's Mac, the Ubuntu ryzen01 box, or lxplus) and sources the matching `thisroot.sh` / `geant4.sh`
- hands off to `common_setup.sh`: CLHEP/Rave/GenFit paths, `LD_LIBRARY_PATH`, `CMAKE_PREFIX_PATH`, `PATH`, and a sanity check for missing/stale variables
- defines `fb` ("faser build"): reruns `cmake -build $HOMEFASER/build -j` from anywhere
- unrecognized machine? it prints exactly which `elif` branch to add
- full from-scratch ROOT/Geant4 install: `docs/INSTALL.md`, `docs/GEANT4_INSTALL.md`

2. Build

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=RelWithDebInfo
cmake --build build -j
```

- first build also compiles CLHEP, Rave, GenFit, googletest, and Pythia8 (CMake superbuild) — slow once, incremental afterward
- after source `setup.sh`, `fb` reruns the build step from anywhere
- executables land in `build/bin/`: `faserps`, `batchreco.exe`, `batchreco_detresp.exe`, `dumphits.exe`, `evDisplay.exe`, `ConvertGENIE.exe`, `Convert.exe`, `CombineFluxes.exe`, the `gtest` binaries, ...

CMake build options

Option	Default	Meaning
FASER_BUILD_EXTERNALS	ON	Build CLHEP/Rave/GenFit/ googletest/Pythia8 from source
FASER_BUILD_GEANT4_PACKAGES	ON	Build FASERG4 and FASERCALProtoG4 (needs Geant4)
FASER_BUILD_DISPLAY	ON	Build the GenFit-based event display / muon spectrometer package
FASER_BUILD_TAUSEARCH	ON	Build TauSearch
FASER_BUILD_TESTS	ON	Build FASERCAL's own gtest suite, registered with CTest

Full list and reuse-an-existing-dependency examples: docs/INSTALL.md

3. Repository layout

FASER/ is the git repo (FASERCAL is this project's own name — see README.md); top level also holds CMakeLists.txt and setup.sh/common_setup.sh.

- CoreUtils/ — core library
- ConvertGENIE/ — GENIE → TPOEvent
- ConvertFASERMC/ — FASERMC → TPOEvent
- ConvertNPZ/ — NPZ conversion
- FASERG4/ — faserps (Geant4 sim)
- FASERCALProtoG4/ — proto calo G4
- Batch/ — batchreco.exe
- Display/ — event display / muon spec
- EvDisplay/ — evDisplay.exe
- Analysis/ — post-reco analysis
- TauSearch/ — tau-neutrino search
- FileMask/ — interaction-type masks
- GeomGDML/ — GDML geometry I/O
- Tests/ — FASERCAL's gtest suite
- cmake/ — CMake helper modules
- docs/ — this deck + docs
- data/ — default \$FASERDATA
- build/ — CMake build dir
- run_*.py, fetch_data.py — wrappers

All gitignored except source; data/, build/ are generated. Analysis/, Display/, EvDisplay/, TauSearch/, FASERCALProtoG4/ each gate on their own FASER_BUILD_* CMake option.

4. \$FASERDATA

- every FASERG4/Batch executable resolves its data location through `$FASERDATA` (`FASER::GetDataDir()`, in `CoreUtils/FaserDataDir.hh`) — not a hardcoded relative path or an inter-directory symlink
- `GetDataDir()` creates a requested subdirectory on demand if it doesn't already exist
- `common_setup.sh` defaults `FASERDATA` to `$HOMEFASER/data` (gitignored) unless a per-site branch in `setup.sh` already exported it elsewhere (scratch, EOS, a dedicated checkout, ...)

\$FASERDATA subdirectories

Subdirectory	Contents	Written by / Read by
faserG4/	Per-event TcalEvent truth files	faserps / batchreco.exe, dumphits.exe, evDisplay.exe
batch/	Reconstructed TPDRcoEvent files	batchreco.exe / analysis code, summarize_output.py
GENIE/	GENIE input samples (too large for git)	fetch_data.py / ConvertGENIE.exe (via run_convertgenie.py)
GDML/	Detector geometry, exported every run	faserps / run_batchreco.py's default --geometry-file
CVGENIE/ Run<run>/	Per-run ConvertGENIE.exe output: PO ROOT files, logs, overlay PNGs	ConvertGENIE.exe (via run_convertgenie.py); then run_faserps.py's --input-file

4. `run_convertgenie.py` — runs ConvertGENIE.exe per detector

- drives ConvertGENIE.exe (`-r <run> -g <gdmlfile> [-l <luminosity_fb>] <genieroot...> [detector]`), auto-discovering its GENIE input files (`$FASERDATA/GENIE/`) and GDML geometry (`$FASERDATA/GDML/`) so normally only `--run` needs to be given by hand
- by default runs **three times** — 3DCAL, ECAL, AHCAL — so all three detectors' PO-format output exists side by side instead of the last run clobbering the others
- output goes to `$FASERDATA/CVGENIE/Run<run>/`: one PO ROOT file per detector, a per-detector log plus a summary log with event counts by interaction category (nueCC, numuCC, nutauCC, NC, ES)

run_convertgenie.py flags

Flag	Default	Meaning
--run	<i>required</i>	Run number, passed as ConvertGENIE.exe -r
--detectors	3DCAL ECAL AHCAL	Which detector(s) to run; also accepts MuonSpec, ALL
--input-files	auto-discovered	GENIE *.gfaser.root file(s)/patterns
--genie-dir	\$FASERDATA/GENIE	Where to auto-discover --input-files from
--geometry-file	auto-discovered	GDMML geometry passed as ConvertGENIE.exe -g
--charmonly / --taucconly	off	Passed through as -charmonly / -tauCConly
--build-dir	build/	CMake build dir containing bin/ConvertGENIE.exe
--output-dir	\$FASERDATA/CVGENIE/ Run<run>	Where PO files, logs, plots are written
--dry-run	off	Print the command(s), don't execute
--force / -f	off	Skip the confirmation prompt, delete+regenerate
--no-overlay	off	Skip the cross-detector overlay-PNG step

run_convertgenie.py behavior

- **luminosity**: auto-detected by scanning the GENIE production's own `run*.sh` script(s) for the `gevgen_faser -l <value>` it was generated with; recorded in the summary log (sets the scale the event counts should be read against) and passed to `ConvertGENIE.exe -l`, which also puts it in the reference-plot titles
- **re-running**: if `--output-dir` already has files in it, asks for confirmation before wiping it and regenerating from scratch (`--force/-f` skips the prompt); either way the directory is removed completely first, so no stale files linger
- **overlay step**: after all detectors finish, runs `ConvertGENIE/make_overlay_plots.C` (skippable with `--no-overlay`; skipped with a warning, not a failure, if `root` isn't on `PATH`) to produce one overlay PNG per quantity with every run detector superimposed

`run_faserps.py` — runs the Geant4 simulation

- builds the V10-geometry macro in memory and pipes it straight to `faserps`' stdin — no `.mac` file to keep in sync
- full prose docs (custom geometry, interactive UI, MuonDIS physics): `run_faserps.md`
- default input sample is auto-discovered from `$FASERDATA/CVGENIE/Run<run>/` (written by `run_convertgenie.py`), prompting for run/detector if more than one is available

run_faserps.py flags (1/2)

Flag	Default	Meaning
--build-dir	build/	CMake build dir containing bin/faserps
--vis	off	Launch the interactive Geant4 UI (no macro used)
--print-macro	off	Print the resolved macro text, exit
--dry-run	off	Print the command, don't execute
--input-file	auto-discovered	ROOT input file; a relative custom value resolves against FASERG4/
--run	auto-discovered	Which CVGENIE/Run<run>/ to read from (skips the run prompt)
--detector	auto-discovered	3DCAL/AHCAL/ ECAL (skips the detector prompt)
--start-event	0	First event index to read
--n-events	100	Number of events (/run/beam0n)
--muons	off	Generate fixed-momentum muons instead of reading --input-file

run_faserps.py flags (2/2)

Flag	Default	Meaning
--muon-momentum-gev	100.0	Muon momentum [GeV]
--muondis	off	Enable on-the-fly Pythia8 DIS per muon interaction (implies --muons)
--muondis-cross-section-bias	150.0	Cross-section bias (1 = unbiased)
--muondis-q2min	1.0	Minimum Q^2 [GeV ²]
--muondis-interaction-log	disabled	CSV path logging each interaction
--muondis-pdf-set	built-in proton PDF	Path under FASERG4/input/
--muondis-xbjmin	0.0	Minimum Bjorken-x cut
--muondis-debug	off	Verbose MuonDIS debug output
--seed	123456789	Geant4 random seed (matches faserps.cc's hardcoded default)
--tilt-deg	-4.5	Detector tilt about Y [deg]
--shift-x-cm / --shift-y-cm	45.0 / 24.0	LOS shift [cm]

run_batchreco.py — runs the reconstruction

- wraps Batch/batchreco.exe with named, validated flags instead of positional arguments
- full prose docs: run_batchreco.md

Flag	Default	Meaning
--run	<i>required</i>	Run number — deliberately no default
--min-event / --max-event	0 / 99999999	Event index range [min, max)
--mask	none (all events)	nueCC, numuCC, nutauCC, nuNC, or nuES
--multi-thread	off	Pass -mt to batchreco.exe
--geometry-file	\$FASERDATA/GDML/ FASERCAL_V10.gdml	GDML geometry to load
--build-dir	build/	CMake build dir containing bin/batchreco.exe
--split	1	Split the range into N parallel background jobs
--dry-run	off	Print the command(s), don't execute

run_regression_tests.py — simulate → reconstruct → compare

- runs the real pipeline end to end for six golden cases (nueCC/numuCC/nutauCC/nuNC, plus --muons and --muondis)
- compares every field (mean_*/rms_*, truth *and* reco) against a committed golden JSON baseline — --rel-tol 10^{-9} by default: near machine precision, *not* a statistical tolerance (this suite expects exact reproducibility, see docs/REGRESSION_TESTS.md)
- batchreco.exe's -mt is **on by default** here (verified against the single-threaded goldens first); raw faserps/batchreco.exe output no longer floods the terminal — it's redirected to work/<case>/logs/*.log

```
python3 run_regression_tests.py --record # first time / deliberate change  
python3 run_regression_tests.py # every other time: compare
```

run_regression_tests.py flags (1/2)

Flag	Default	Meaning
<code>--case</code>	all cases	Restrict to a golden case name (repeatable); see <code>--list</code>
<code>--record</code>	off	Write the golden file(s) instead of comparing
<code>--rel-tol</code>	10^{-9}	Relative tolerance for floating-point comparisons — near machine precision, not a statistical tolerance
<code>--build-dir</code>	build/	CMake build directory
<code>--python</code>	<code>\$FASER_PYTHON</code> env, else current	Interpreter for the subprocess calls — needs a PyROOT-enabled one matching your ROOT build
<code>--list</code>	—	List available case names, exit

run_regression_tests.py flags (2/2)

Flag	Default	Meaning
<code>--skip-faserps</code>	off	Reuse the truth sample already in <code>work/<case>/data/faserG4</code> instead of re-simulating
<code>--skip-reco</code>	off	Reuse the reco file already in <code>work/<case>/data/batch</code> instead of re-running <code>batchreco.exe</code> ; combine with <code>--skip-faserps</code> to jump straight to summarize
<code>--multi-thread</code> / <code>--no-multi-thread</code>	on / off	Pass <code>batchreco.exe</code> 's <code>-mt</code> (on by default here, unlike <code>run_batchreco.py</code> standalone)

Tests/regression/results/<case>.json (the real numbers) and, for a plain comparison, results/comparison.csv — one row per field per case: golden, current, abs_diff, rel_diff, status. See docs/REGRESSION_TESTS.md's "Example: what a typical run looks like".

Golden JSON schema

One file per case (Tests/regression/golden/<case>.json): meta records how it was produced; truth/reco are aggregate numbers only — every mean_* has an rms_* partner (ROOT's TH1::GetRMS() convention) — not full per-event dumps, so a diff stays readable in a PR. Real content of golden/neutrino_numuCC.json (rounded/trimmed for space):

```
{
  "meta": {"case": "neutrino_numuCC", "reaction": "numuCC",
    "faserps_args": [], "run_number": 10000,
    "n_events_simulated": 100},
  "truth": {"n_events": 25,
    "mean_Evis": 595.73, "rms_Evis": 401.11,
    "mean_Q2": 77.26, "rms_Q2": 58.73,
    "mean_xBj": 0.202, "rms_xBj": 0.140, ...},
  "reco": {"n_events_reconstructed": 25,
    "mean_n_POREcs": 14.76, "rms_n_POREcs": 4.91,
    "mean_total_Ecompensated": 72.63,
    "rms_total_Ecompensated": 66.27, ...}
}
```

n_events/n_events_reconstructed are how many of the 100 simulated events were actually this reaction (unbiased sample, see above) — not 100. muons/muondis omit the DIS fields entirely where meaningless, rather than padding with zeros.

Example: what a typical run looks like

```
[run_regression_tests] neutrino: faserps: ... --input-file ... --n-events 100 ...
[run_regression_tests] neutrino: faserps: output -> work/neutrino/logs/faserps.log
[run_regression_tests] neutrino: batchreco: ... --multi-thread
[run_regression_tests] neutrino: batchreco: output -> work/neutrino/logs/batchreco.log
[run_regression_tests] neutrino_nueCC: PASS
[run_regression_tests] neutrino_numuCC: PASS
[run_regression_tests] neutrino_nutauCC: PASS
[run_regression_tests] neutrino_nuNC: PASS
[run_regression_tests] muons: PASS
[run_regression_tests] muondis: PASS
[run_regression_tests] full per-case results ... written under Tests/regression/results/
[run_regression_tests] field-by-field comparison ... (148 fields, 0 FAIL) written to
Tests/regression/results/comparison.csv
```

Exit code 0 (every case PASS). A real FAIL example and real comparison.csv rows (golden, current, abs_diff, rel_diff, status — per field, not just the FAILs): docs/REGRESSION_TESTS.md, §“Example: what a typical run looks like”.

fetch_data.py & summarize_output.py

`fetch_data.py` — public-CERNBox-link downloads into `$FASERDATA`, sha256-verified. Manual step, not auto-triggered (the old auto-fetch-on-missing-file path was removed when GENIE/ became CVGENIE/); covers only the *raw* GENIE output, not the converted PO sample — see `docs/REGRESSION_TESTS.md`'s open items.

Flag	Default	Meaning
<code>--force</code>	off	Re-download even if already present
<code>--dry-run</code>	off	Show what would be fetched, don't fetch
<code>--list</code>	—	List manifest entries, exit

`Tests/regression/summarize_output.py` — PyROOT summarizer used internally by `run_regression_tests.py` (requires `--truth-dir` and/or `--reco-file`).

Flag	Default	Meaning
<code>--dict-path</code>	CoreUtils' built dict	Path to the built ROOT dictionary
<code>--truth-dir</code>	none (skip)	Dir of truth files; needs <code>--run</code> , <code>--n-events</code>
<code>--reco-file</code>	none (skip)	Path to one Batch-TP0Recevent_*.root file

6. A typical end-to-end run

```
source setup.sh
cmake -S . -B build -DCMAKE_BUILD_TYPE=RelWithDebInfo && cmake --build build -j

python3 run_faserps.py --n-events 500 # simulate
python3 run_batchreco.py --run 10000 --mask numuCC # reconstruct numuCC
build/bin/evDisplay.exe 10000 numuCC # look at the result
```

or, to exercise the whole pipeline and check it against the committed baseline:

```
python3 run_regression_tests.py
```

7. Tests

```
ctest --test-dir build --output-on-failure  
ctest --test-dir build -R Magnet --output-on-failure # run a subset by name
```

- FASERCAL's own gtest suite (MuonSpectrometerFieldTest, MagnetGeometryProbeTest, ...) builds automatically whenever FASER_BUILD_TESTS and FASER_BUILD_GEANT4_PACKAGES are both ON (the default)
- the heavier simulate-reconstruct-compare regression suite is run_regression_tests.py, described above and in docs/REGRESSION_TESTS.md

Adding a new test

- 1 write Tests/MyNewTest.cc:

```
#include <gtest/gtest.h>
TEST(MySuite, MyCase) { EXPECT_EQ(2 + 2, 4); }
```

- 2 add a matching add_executable() / target_link_libraries() / add_test() block to Tests/CMakeLists.txt, modeled on MuonSpectrometerFieldTest / MagnetGeometryProbeTest — there's no automatic *Test.cc discovery, each test needs its own block
- 3 if it links Geant4, it also needs the find_package(Geant4 ...) already at the top of that file, plus the same -Wl,--allow-shlib-undefined Linux workaround the existing Geant4-linking tests use
- 4 reconfigure once (cmake -S . -B build && cmake -build build -j, or fb) so CMake picks up the new target — ctest then finds it automatically

Full walkthrough: docs/HOWTO.md, §7.

- `docs/INSTALL.md` — the full CMake build: prerequisites, every option, troubleshooting a clean build
- `docs/GEANT4_INSTALL.md` — building/installing Geant4 itself
- `run_faserps.md` / `run_batchreco.md` — prose documentation for those two wrappers
- `docs/README_MuonDIS.md` — MuonDIS physics and every `/physics/muondis/...` option
- `docs/REGRESSION_TESTS.md` — the regression suite's full design, what's verified vs. not, and open items